

```
package org.prateek.BehaviouralPatterns.ChainOfResponsibilityPattern;

public class Client {
    public static void main(String[] args) {
        Handler supportAgent = new SupportAgent();
        Handler supervisor = new Supervisor();
        Handler manager = new Manager();
        Handler director = new Director();

        supportAgent.setNext(supervisor);
        supervisor.setNext(manager);
        manager.setNext(director);

        Request technicalRequest = new Request("Technical", "Cannot login
to account", 1);
        Request billingRequest = new Request("Billing", "Invoice
discrepancy", 2);
        Request refundRequest = new Request("Refund", "Request for full
refund", 4);

        System.out.println("Processing Technical Request");
        supportAgent.handleRequest(technicalRequest);
        System.out.println();

        System.out.println("Processing Billing Request");
        supportAgent.handleRequest(billingRequest);
        System.out.println();

        System.out.println("Processing Refund Request");
        supportAgent.handleRequest(refundRequest);
    }
}
```

```
package org.prateek.BehaviouralPatterns.ChainOfResponsibilityPattern;

public class Director extends Handler {
    @Override
    public void handleRequest(Request request) {
        System.out.println("Director handling: " +
request.getDescription());
    }
}
```

```
package org.prateek.BehaviouralPatterns.ChainOfResponsibilityPattern;

public abstract class Handler {
    protected Handler nextHandler;

    public void setNext(Handler nextHandler) {
        this.nextHandler = nextHandler;
    }

    public abstract void handleRequest(Request request);
}
```

```
package org.prateek.BehaviouralPatterns.ChainOfResponsibilityPattern;

public class Manager extends Handler {
    @Override
    public void handleRequest(Request request) {
        if (request.getPriority() <= 3) {
            System.out.println("Manager handling: " +
request.getDescription());
        } else {
            if (nextHandler != null) {
                nextHandler.handleRequest(request);
            }
        }
    }
}
```

```
package org.prateek.BehaviouralPatterns.ChainOfResponsibilityPattern;

public class Request {
    private String type;
    private String description;
    private int priority;

    public Request(String type, String description, int priority) {
        this.type = type;
        this.description = description;
        this.priority = priority;
    }

    public String getType() {
        return type;
    }

    public String getDescription() {
        return description;
    }

    public int getPriority() {
        return priority;
    }
}
```

```
package org.prateek.BehaviouralPatterns.ChainOfResponsibilityPattern;

public class Supervisor extends Handler {
    @Override
    public void handleRequest(Request request) {
        if (request.getPriority() <= 2) {
            System.out.println("Supervisor handling: " +
request.getDescription());
        } else {
            if (nextHandler != null) {
                nextHandler.handleRequest(request);
            }
        }
    }
}
```

```
package org.prateek.BehaviouralPatterns.ChainOfResponsibilityPattern;

public class SupportAgent extends Handler {
    @Override
    public void handleRequest(Request request) {
        if (request.getPriority() <= 1) {
            System.out.println("Support Agent handling: " +
request.getDescription());
        } else {
            if (nextHandler != null) {
                nextHandler.handleRequest(request);
            }
        }
    }
}
```

```
package org.prateek.BehaviouralPatterns.CommandPattern;
```

```
//Command Interface
```

```
interface Command {  
    void execute();  
}
```

```
//Concrete Classes for Commands
```

```
class BoldCommand implements Command{  
    private TextEditorII editor;  
  
    public BoldCommand(TextEditorII editor){  
        this.editor = editor;  
    }  
  
    @Override  
    public void execute() {  
        editor.boldText();  
    }  
}
```

```
class ChangeColor implements Command{  
    private TextEditorII editor;  
  
    public ChangeColor(TextEditorII editor){  
        this.editor = editor;  
    }  
  
    @Override  
    public void execute() {  
        editor.changeColor();  
    }  
}
```

```
//Button Class
```

```
class Button{  
    private Command command;  
  
    public void setCommand(Command command){  
        this.command = command;  
    }  
    public void click(){  
        command.execute();  
    }  
}
```

```
// Receiver: TextEditor class
```

```
class TextEditorII {  
    public void boldText() {  
        System.out.println("Text has been bolded.");  
    }  
  
    public void italicizeText() {
```

```

        System.out.println("Text has been italicized.");
    }

    public void underlineText() {
        System.out.println("Text has been underlined.");
    }

    public void changeColor(){
        System.out.println("Text color has been changed");
    }
}

public class WithCommandPattern {
    public static void main(String[] args) {
        TextEditorII editorII = new TextEditorII();

        //Button
        // Decoupling -> One button can do many types of actions, decoupled
from the text editor!
        Button button = new Button();
        button.setCommand(new BoldCommand(editorII));
        button.setCommand(new ChangeColor(editorII));
        button.click();

    }
}

```

```

package org.prateek.BehaviouralPatterns.CommandPattern;

class TextEditor {
    public void boldText() {
        System.out.println("Text has been bolded.");
    }

    public void italicizeText() {
        System.out.println("Text has been italicized.");
    }

    public void underlineText() {
        System.out.println("Text has been underlined.");
    }
}

//UI Button Classes
class BoldButton{
    private TextEditor editor;
    public BoldButton(TextEditor editor){
        this.editor = editor;
    }
    public void click(){
        editor.boldText();
    }
}
class ItalicButton {
    private TextEditor editor;

    public ItalicButton(TextEditor editor) {
        this.editor = editor;
    }

    public void click() {
        editor.italicizeText();
    }
}

public class WithoutCommandPattern {
    public static void main(String[] args) {
        TextEditor editor = new TextEditor();
        BoldButton boldButton = new BoldButton(editor);
        ItalicButton italicButton = new ItalicButton(editor);
        boldButton.click();
        italicButton.click();
    }
}

```

```
package org.prateek.BehaviouralPatterns.IteratorPattern;

public class Book implements Comparable<Book>{
    private String title;

    public Book(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }

    @Override
    public String toString() {
        return "Book{" +
            "title='" + title + '\\\'' +
            "}'";
    }

    @Override
    public int compareTo(Book o) {
        return this.title.compareTo(o.title);
    }
}
```

```
package org.prateek.BehaviouralPatterns.IteratorPattern;

import java.util.ArrayList;
import java.util.List;

public class BookCollectionV1 {
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        books.add(book);
    }

    public List<Book> getBooks() {
        return books;
    }
}
```

```

package org.prateek.BehaviouralPatterns.IteratorPattern;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class BookCollectionV2 {
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        books.add(book);
    }

    public List<Book> getBooks() {
        return books;
    }

    public Iterator<Book> createIterator() {
        return new BookIterator(this.books);
    }

    //Another Class (Nested Class)
    private class BookIterator implements Iterator<Book>{
        private List<Book> books;
        private int position = 0;

        public BookIterator(List<Book> books) {
            this.books = books;
        }

        @Override
        public boolean hasNext() {
            return position < books.size();
        }

        @Override
        public Book next() {
            return books.get(position++);
        }
    }
}

```

```
package org.prateek.BehaviouralPatterns.IteratorPattern;

import com.sun.source.tree.Tree;

import java.util.*;

public class BookCollectionV3 implements Iterable<Book>{
    private Set<Book> books = new TreeSet<>();

    public void addBook(Book book) {
        books.add(book);
    }

    @Override
    public Iterator<Book> iterator() {
        return books.iterator();
    }
}
```

```
package org.prateek.BehaviouralPatterns.IteratorPattern;

public class ClientV1 {
    public static void main(String[] args) {
        BookCollectionV1 bookCollection = new BookCollectionV1();
        bookCollection.addBook(new Book("C++ Book"));
        bookCollection.addBook(new Book("Java Book"));
        bookCollection.addBook(new Book("Python Book"));

        for(int i=0;i<bookCollection.getBooks().size();i++){
            System.out.println(bookCollection.getBooks().get(i));
        }
    }
}
```

```
package org.prateek.BehaviouralPatterns.IteratorPattern;

import java.util.Iterator;

public class ClientV2 {
    public static void main(String[] args) {
        BookCollectionV2 bookCollection = new BookCollectionV2();
        bookCollection.addBook(new Book("C++ Book"));
        bookCollection.addBook(new Book("Java Book"));
        bookCollection.addBook(new Book("Python Book"));

        Iterator<Book> iterator = bookCollection.createIterator();
        while(iterator.hasNext()){
            Book book = iterator.next();
            System.out.println(book);
        }
    }
}
```

```
package org.prateek.BehaviouralPatterns.IteratorPattern;

import java.util.Iterator;

public class ClientV3 {
    public static void main(String[] args) {
        BookCollectionV3 bookCollection = new BookCollectionV3();
        bookCollection.addBook(new Book("Python Book"));
        bookCollection.addBook(new Book("C++ Book"));
        bookCollection.addBook(new Book("Java Book"));

        Iterator<Book> iterator = bookCollection.iterator(); //Standardized

        while(iterator.hasNext()){
            Book book = iterator.next();
            System.out.println(book);
        }
    }
}
```

```

package org.prateek.BehaviouralPatterns.MediatorPattern;

import java.util.ArrayList;
import java.util.List;

interface ChatMediator{
    void sendMessage(String msg,ChatUser user);
    void addUser(ChatUser user);
}

class ChatRoom implements ChatMediator{
    private List<ChatUser> users;

    public ChatRoom(){
        this.users = new ArrayList<>();
    }

    @Override
    public void sendMessage(String msg, ChatUser sender) {
        for(ChatUser user: users){
            if(user!=sender){
                user.recieveMessage(msg, sender);
            }
        }
    }

    @Override
    public void addUser(ChatUser user) {
        users.add(user);
    }
}

class ChatUser{
    private String name;
    private ChatMediator chatMediator;

    public ChatUser(String name,ChatMediator chatMediator){
        this.name = name;
        this.chatMediator = chatMediator;
    }

    public void sendMessage(String msg){
        System.out.println(this.name + "Sending a msg " + msg);
        chatMediator.sendMessage(msg,this);
    }

    public String getName(){
        return name;
    }

    public void recieveMessage(String msg, ChatUser sender){
        System.out.println(this.name + " received message: '" + msg + "'
from " + sender.getName());
    }
}

```

```
    }  
}  
  
public class WithMediatorPattern {  
    public static void main(String[] args) {  
        ChatMediator chatRoom = new ChatRoom();  
        ChatUser rahul = new ChatUser("Rahul", chatRoom);  
        ChatUser amit = new ChatUser("Amit", chatRoom);  
        ChatUser neha = new ChatUser("Neha", chatRoom);  
  
        chatRoom.addUser(rahul);  
        chatRoom.addUser(neha);  
        chatRoom.addUser(amit);  
  
        amit.sendMessage("Hi everyone");  
    }  
}
```

```
package org.prateek.BehaviouralPatterns.MediatorPattern;

//Chat System
class User{
    private String name;

    public User(String name){
        this.name = name;
    }

    public void sendMessage(String msg, User recipient){
        System.out.println(this.name + " sending "+ msg + " to " +
recipient.getName());
    }

    public String getName(){
        return name;
    }
}

public class WithoutMediatorPattern {
    public static void main(String[] args) {
        User rahul = new User("Rahul");
        User amit = new User("Amit");
        User neha = new User("Neha");

        rahul.sendMessage("Hello", amit);
        rahul.sendMessage("Hello", neha);

    }
}
```

```
package org.prateek.BehaviouralPatterns.MementoPattern;

import java.util.Stack;

//Caretaker Class: Manage mementos (snapshots of the TextEditor state)
public class Caretaker {
    private final Stack<EditorMemento> history = new Stack<>();

    public void saveState(TextEditor editor){
        history.push(editor.save());
    }
    public void undo(TextEditor editor){
        if(!history.empty()){
            history.pop();
            editor.restore(history.peek());
        }
    }
}
```

```
package org.prateek.BehaviouralPatterns.MementoPattern;
```

```
//Mementor Class: Stores the internal state of the TextEditor
```

```
public class EditorMemento {  
    private final String content;  
  
    public EditorMemento(String content) {  
        this.content = content;  
    }  
  
    public String getContent(){  
        return content;  
    }  
}
```

```
package org.prateek.BehaviouralPatterns.MementoPattern;

/* A text editor where the user can undo changes, such as text addition,
deletion,
or formatting. The editor stores snapshots of its state (text content)
after each change, enabling the user to revert to previous states.
*/
public class TextEditor {
    private String content;
    public void write(String text){
        this.content = text;
    }
    //Save the current state of editor
    public EditorMemento save(){
        return new EditorMemento(content);
    }

    //Restore (Memento->update the state of current content)
    public void restore(EditorMemento memento){
        content = memento.getContent();
    }

    public String getContent(){
        return content;
    }
}
```

```
package org.prateek.BehaviouralPatterns.MementoPattern;

public class TextEditorMain {
    public static void main(String[] args) {
        TextEditor editor = new TextEditor();
        Caretaker caretaker = new Caretaker(); //History / StateMgmt

        editor.write("A");
        caretaker.saveState(editor);

        editor.write("B");
        caretaker.saveState(editor);
        //Problem - > Undo the last write!
        editor.write("C");
        caretaker.saveState(editor);

        //CareTaker Undo
        caretaker.undo(editor);

        System.out.println(editor.getContent());
    }
}
```

```

package org.prateek.BehaviouralPatterns.ObserverPattern;

import java.util.ArrayList;
import java.util.List;

//Observer Interface
interface Observer{
    void update(float temp);
}

//Subject Interface
interface Subject{
    void attach(Observer obs);
    void detach(Observer obs);
    void notifyObservers();
}

// Weather Station
class WeatherStationIn implements Subject{
    private float temperature;
    private List<Observer> observerList;

    public WeatherStationIn(){
        observerList = new ArrayList<>(); //Empty
    }

    public void setTemperature(float temperature){
        this.temperature = temperature;
        notifyObservers();
    }

    @Override
    public void attach(Observer obs) {
        observerList.add(obs);
    }

    @Override
    public void detach(Observer obs) {
        observerList.remove(obs);
    }

    @Override
    public void notifyObservers() {
        for(Observer obs:observerList){
            obs.update(temperature); //Run Time Polymorphism
        }
    }
}

class DisplayDeviceIn implements Observer{
    String name;
    public DisplayDeviceIn(String deviceName){
        this.name = deviceName;
    }
}

```

```

    }
    @Override
    public void update(float temp) {
        System.out.println("Temp on " + name + " device is " + temp);
    }
}

class MobileDevice implements Observer{
    @Override
    public void update(float temp) {
        System.out.println("Temp on mobile is " + temp);
    }
}

public class ObserverPatternExample {
    public static void main(String[] args) {
        //Create a Publisher
        WeatherStationIn weatherStationIn = new WeatherStationIn();

        //Create subscribers
        DisplayDeviceIn device = new DisplayDeviceIn("SamsungLCD");
        MobileDevice mobileDevice = new MobileDevice();

        //Attach
        weatherStationIn.attach(device);
        weatherStationIn.attach(mobileDevice);

        //Set Temp
        weatherStationIn.setTemperature(25);

        //Detach
        weatherStationIn.detach(mobileDevice);

        weatherStationIn.setTemperature(26);
    }
}

```

```

package org.prateek.BehaviouralPatterns.ObserverPattern;

class DisplayDevice{
    public void showTemp(float temp){
        System.out.println("Current Temp: " + temp + " C");
    }
}

class WeatherStation{
    private float temperature;
    private DisplayDevice displayDevice; //can be multiple such devices
    later on

    public WeatherStation(DisplayDevice displayDevice){
        this.displayDevice = displayDevice;
    }

    public void setTemperature(float temp){
        this.temperature = temp;
        notifyDevice();
    }
    public void notifyDevice(){
        displayDevice.showTemp(temperature);
    }
}

public class WithoutObserverPattern {
    public static void main(String[] args) {
        DisplayDevice device = new DisplayDevice();
        WeatherStation station = new WeatherStation(device);
        //Tight Coupling B/w station and the device
        station.setTemperature(26);
        station.setTemperature(30);
    }
}

```

```
package org.prateek.BehaviouralPatterns.StatePattern.Solution;

public class WithStatePattern {
    public static void main(String[] args) {
        DirectionService service = new DirectionService(new Car());
        // service.setTransportationMode(new Cycling());

        System.out.println("ETA: " + service.getETA());
        System.out.println("Direction: " + service.getDirection());

    }
}
```

```
package org.prateek.BehaviouralPatterns.StatePattern.Solution;

public class Walking implements TransportationMode{
    @Override
    public int calcETA() {
        System.out.println("Calculating ETA (Walking)");
        return 10; // Example ETA for walking
    }

    @Override
    public String getDirection() {
        return "Directions for walking";
    }
}
```

```
package org.prateek.BehaviouralPatterns.StatePattern.Solution;

//State Interface
public interface TransportationMode {
    int calcETA();
    String getDirection();
}
```

```
package org.prateek.BehaviouralPatterns.StatePattern.Solution;

//

public class Train implements TransportationMode {
    @Override
    public int calcETA() {
        System.out.println("Calculating ETA (Train)");
        return 7; // Example ETA for train
    }

    @Override
    public String getDirection() {
        return "Directions for train route";
    }
}
```

```
package org.prateek.BehaviouralPatterns.StatePattern.Solution;

public class DirectionService {
    TransportationMode transportationMode;
    public DirectionService(TransportationMode transportationMode){
        this.transportationMode = transportationMode;
    }
    public void setTransportationMode(TransportationMode mode){
        this.transportationMode = mode;
    }
    //delegating the work current state's concrete class
    public int getETA(){
        return transportationMode.calcETA();
    }
    public String getDirection(){
        return transportationMode.getDirection();
    }
}
```

```
package org.prateek.BehaviouralPatterns.StatePattern.Solution;

public class Cycling implements TransportationMode{
    @Override
    public int calcETA() {
        System.out.println("Calculating ETA (Cycling)");
        return 5; // Example ETA for cycling
    }

    @Override
    public String getDirection() {
        return "Directions for cycling";
    }
}
```

```
package org.prateek.BehaviouralPatterns.StatePattern.Solution;

public class Car implements TransportationMode {
    @Override
    public int calcETA() {
        System.out.println("Calculating ETA (Car)");
        return 3; // Example ETA for car
    }

    @Override
    public String getDirection() {
        return "Directions for driving";
    }
}
```

```
package org.prateek.BehaviouralPatterns.StatePattern.Problem;

public class WithoutStatePattern {
    public static void main(String[] args) {
        DirectionService directionService = new
DirectionService(TransportationMode.TRAIN);
        directionService.setMode(TransportationMode.CAR);

        System.out.println(directionService.getDirection());
        directionService.getETA();
    }
}
```

```

package org.prateek.BehaviouralPatterns.StatePattern.Problem;

enum TransportationMode{
    WALKING, CYCLING, CAR, TRAIN
}

public class DirectionService {
    private TransportationMode mode;

    public DirectionService(TransportationMode mode){
        this.mode = mode;
    }

    public void setMode(TransportationMode mode){
        this.mode = mode;
    }

    //Method to calc ETA based upon transportation Mode
    public int getETA() {
        switch (mode){
            case WALKING:
                System.out.println("Calc ETA for walking 10");
                return 10;
            case CYCLING:
                System.out.println("Calc ETA for cycling 5");
                return 5;
            case CAR:
                System.out.println("Calc ETA for car 2");
                return 2;
            case TRAIN:
                System.out.println("Calc ETA for cycling 3");
                return 3;
            default:
                throw new IllegalArgumentException("Unknown Mode");
        }
    }

    public String getDirection() {
        switch (mode) {
            case WALKING:
                return "Directions for walking: Head north for 500
meters.";
            case CYCLING:
                return "Directions for cycling: Take the bike lane on Elm
Street.";
            case CAR:
                return "Directions for driving: Use highway 50 towards
downtown.";
            case TRAIN:
                return "Directions for train: Board the Red Line at Central
Station.";
            default:
                return "No directions available for the selected mode.";
        }
    }
}

```



```
package org.prateek.BehaviouralPatterns.StrategyPattern;

class PaymentService{
    public void processPayment (String paymentMethod) {
        if(paymentMethod.equals("Credit Card")){
            System.out.println("Making payment via credit card");
        }
        else if(paymentMethod.equals("Debit Card")){
            System.out.println("Making payment via debit card");
        }
        else if(paymentMethod.equals("UPI")){
            //huge algorithm
            System.out.println("Making payment via UPI");
        }
        else{
            System.out.println("Unsupported Payment method");
        }
    }
}

public class WithoutStrategyPattern {
    public static void main(String[] args) {
        PaymentService paymentService = new PaymentService();
        paymentService.processPayment ("UPI");
    }
}
```

```

package org.prateek.BehaviouralPatterns.StrategyPattern;

interface PaymentStrategy{
    void processPayment();
}
//Concrete Strategy: Credit Card
class CreditCardPayment implements PaymentStrategy{

    @Override
    public void processPayment() {
        System.out.println("Making payment via Credit Card");
    }
}
//Concrete Strategy: Debit Card
class DebitCardPayment implements PaymentStrategy{

    @Override
    public void processPayment() {
        System.out.println("Making payment via Debit Card");
    }
}

class UPIPayment implements PaymentStrategy{

    @Override
    public void processPayment() {
        System.out.println("Making payment via UPI");
    }
}

class PaymentServiceII{
    private PaymentStrategy strategy;
    public void setPaymentStrategy(PaymentStrategy strategy){
        this.strategy = strategy;
    }

    public void pay(){
        strategy.processPayment(); //Polymorphic Behaviour
    }
}

public class StrategyPattern {
    public static void main(String[] args) {
        PaymentServiceII paymentService = new PaymentServiceII();
        paymentService.setPaymentStrategy(new UPIPayment());
        paymentService.pay();
    }
}

```

```

package org.prateek.BehaviouralPatterns.TemplateMethodPattern;

import javax.xml.crypto.Data;

abstract class DataParser{

    //Template Method Define the steps of the algorithm
    public final void parse() {
        openFile();
        parseData();
        closeFile();
    }
    protected void openFile(){
        System.out.println("Opening File");
    }
    protected void closeFile(){
        System.out.println("Closing File");
    }
    protected abstract void parseData();
}
//CSV Parser
class CSVParserII extends DataParser{

    @Override
    protected void parseData() {
        System.out.println("Parsing CSV Data");
    }
}
class JSONParserII extends DataParser{

    @Override
    protected void parseData() {
        System.out.println("Parsing JSON Data");
    }
}

public class WithTemplatePattern {
    public static void main(String[] args) {
        DataParser csvParser = new CSVParserII();
        DataParser jsonParser = new JSONParserII();
        csvParser.parse();
        jsonParser.parse();
    }
}

```

```
package org.prateek.BehaviouralPatterns.TemplateMethodPattern;
```

```
//CSV Parser
```

```
class CSVParser{  
    public void parse(){  
        openFile();  
        //CSV Specific Parsing Logic  
        System.out.println("Parsing a CSV File");  
        closeFile();  
    }  
    private void openFile(){  
        System.out.println("Opening File");  
    }  
    private void closeFile(){  
        System.out.println("Closing File");  
    }  
}
```

```
class JSONParser{  
    public void parse(){  
        openFile();  
        //CSV Specific Parsing Logic  
        System.out.println("Parsing a JSON File");  
        closeFile();  
    }  
    private void openFile(){  
        System.out.println("Opening File");  
    }  
    private void closeFile(){  
        System.out.println("Closing File");  
    }  
}
```

```
public class WithoutTemplateMethod {  
    public static void main(String[] args) {  
        CSVParser csvParser = new CSVParser();  
        csvParser.parse();  
        JSONParser jsonParser = new JSONParser();  
        jsonParser.parse();  
    }  
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;
```

```
public class WordDocument implements Document {  
    private String fileName;  
    private String author;  
  
    public WordDocument(String fileName, String author) {  
        this.fileName = fileName;  
        this.author = author;  
    }  
  
    @Override  
    public void accept(DocumentVisitor visitor) {  
        visitor.visitWord(this);  
    }  
  
    public String getFileName() {  
        return fileName;  
    }  
  
    public String getAuthor() {  
        return author;  
    }  
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;
```

```
public class TextDocument implements Document {  
    private String fileName;  
    private int lineCount;  
  
    public TextDocument(String fileName, int lineCount) {  
        this.fileName = fileName;  
        this.lineCount = lineCount;  
    }  
  
    @Override  
    public void accept(DocumentVisitor visitor) {  
        visitor.visitText(this);  
    }  
  
    public String getFileName() {  
        return fileName;  
    }  
  
    public int getLineCount() {  
        return lineCount;  
    }  
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;

public class PrintVisitor implements DocumentVisitor {

    @Override
    public void visitPDF(PDFDocument document) {
        System.out.println("Printing PDF: " + document.getFileName());
    }

    @Override
    public void visitWord(WordDocument document) {
        System.out.println("Printing Word: " + document.getFileName());
    }

    @Override
    public void visitText(TextDocument document) {
        System.out.println("Printing Text: " + document.getFileName());
    }
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;

public class PDFDocument implements Document {
    private String fileName;
    private int pageCount;

    public PDFDocument(String fileName, int pageCount) {
        this.fileName = fileName;
        this.pageCount = pageCount;
    }

    @Override
    public void accept(DocumentVisitor visitor) {
        visitor.visitPDF(this);
    }

    public String getFileName() {
        return fileName;
    }

    public int getPageCount() {
        return pageCount;
    }
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;

public class ExportVisitor implements DocumentVisitor {

    @Override
    public void visitPDF(PDFDocument document) {
        System.out.println("Exporting PDF: " + document.getFileName());
    }

    @Override
    public void visitWord(WordDocument document) {
        System.out.println("Exporting Word: " + document.getFileName());
    }

    @Override
    public void visitText(TextDocument document) {
        System.out.println("Exporting Text: " + document.getFileName());
    }
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;

public interface DocumentVisitor {
    void visitPDF(PDFDocument document);
    void visitWord(WordDocument document);
    void visitText(TextDocument document);
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;

public interface Document {
    void accept(DocumentVisitor visitor);
}
```

```
package org.prateek.BehaviouralPatterns.VisitorPattern;

public class CompressVisitor implements DocumentVisitor {
    @Override
    public void visitPDF(PDFDocument document) {
        System.out.println("Compressing PDF: " + document.getFileName());
    }
    @Override
    public void visitWord(WordDocument document) {
        System.out.println("Compressing Word: " + document.getFileName());
    }
    @Override
    public void visitText(TextDocument document) {
        System.out.println("Compressing Text: " + document.getFileName());
    }
}
```

```

package org.prateek.BehaviouralPatterns.VisitorPattern;

import java.util.ArrayList;
import java.util.List;

public class Client {
    public static void main(String[] args) {
        PDFDocument pdfDoc = new PDFDocument("report.pdf", 25);
        WordDocument wordDoc = new WordDocument("letter.docx", "John Doe");
        TextDocument textDoc = new TextDocument("notes.txt", 3);

        List<Document> documents = new ArrayList<>();
        documents.add(pdfDoc);
        documents.add(wordDoc);
        documents.add(textDoc);

        System.out.println("Export Operation");
        DocumentVisitor exportVisitor = new ExportVisitor();
        for (Document doc : documents) {
            doc.accept(exportVisitor);
        }
        System.out.println();

        System.out.println("Print Operation");
        DocumentVisitor printVisitor = new PrintVisitor();
        for (Document doc : documents) {
            doc.accept(printVisitor);
        }
        System.out.println();

        System.out.println("Compress Operation");
        DocumentVisitor compressVisitor = new CompressVisitor();
        for (Document doc : documents) {
            doc.accept(compressVisitor);
        }
    }
}

```

```
package org.prateek.CreationalPatterns.AbstractFactoryPattern.Solution;
```

```
//Abstract Product Interfaces
```

```
interface Button{  
    void render();  
}
```

```
interface ScrollBar{  
    void scroll();  
}
```

```
//Windows UI Components
```

```
class WindowsButton implements Button{  
    public void render(){  
        System.out.println("Rendering windows button");  
    }  
}
```

```
class WindowsScrollBar implements ScrollBar{  
    public void scroll(){  
        System.out.println("Scrolling windows scrollbar");  
    }  
}
```

```
//Mac UI components
```

```
class MacOSButton implements Button{  
    public void render(){  
        System.out.println("Rendering MacOS button");  
    }  
}
```

```
class MacOSScrollBar implements ScrollBar{  
    public void scroll(){  
        System.out.println("Scrolling MacOS scrollbar");  
    }  
}
```

```
//Abstract Factory Interface
```

```
interface UIFactory{  
    Button createButton();  
    ScrollBar createScrollBar();  
}
```

```
//Concrete Implementation
```

```
class WindowsFactory implements UIFactory{
```

```
    @Override  
    public Button createButton() {  
        return new WindowsButton();  
    }
```

```
    @Override  
    public ScrollBar createScrollBar() {  
        return new WindowsScrollBar();  
    }  
}
```

```

}
class MacOSFactory implements UIFactory{

    @Override
    public Button createButton() {
        return new MacOSButton();
    }

    @Override
    public ScrollBar createScrollBar() {
        return new MacOSScrollBar();
    }
}

public class Application {
    private Button button;
    private ScrollBar scrollBar;
    public Application(UIFactory factory){
        this.button = factory.createButton();
        this.scrollBar = factory.createScrollBar();
    }

    public void renderUI(){
        button.render();
        scrollBar.scroll();
    }

    public static void main(String[] args) {
        //Use Windows UI
        UIFactory windowsFactory = new WindowsFactory();
        Application app = new Application(new MacOSFactory());
        app.renderUI();
    }
}

```

```
package org.prateek.CreationalPatterns.AbstractFactoryPattern.Problem;
//Windows UI Components

class WindowsButton{
    public void render(){
        System.out.println("Rendering windows button");
    }
}
class WindowsScrollBar{
    public void render(){
        System.out.println("Rendering windows scrollbar");
    }
}

//Mac UI components
class MacOSButton{
    public void render(){
        System.out.println("Rendering MacOS button");
    }
}
class MacOSScrollBar{
    public void render(){
        System.out.println("Rendering MacOS scrollbar");
    }
}

public class Application {
    public static void main(String[] args) {
        //Windows UI
        WindowsButton button = new WindowsButton();
        WindowsScrollBar scrollBar = new WindowsScrollBar();
        //Mac UI
        button.render();
        scrollBar.render();
    }
}
```

```
package org.prateek.CreationalPatterns.BuilderPattern.Solution;

public class WithBuilderPattern {
    public static void main(String[] args) {
        House house = new House.HouseBuilder("Concrete", "Wooden", "Tile")
            .setGarden(true)
            .setSwimmingPool(true)
            .setGarage(false)
            .build();

        System.out.println(house);
    }
}
```

```

package org.prateek.CreationalPatterns.BuilderPattern.Solution;

public class House {
    private String foundation;
    private String structure;
    private String roof;
    private boolean hasGarage;
    private boolean hasSwimmingPool;
    private boolean hasGarden;

    //Constructor
    private House(HouseBuilder builder) {
        this.foundation = builder.foundation;
        this.structure = builder.structure;
        this.roof = builder.roof;
        this.hasGarage = builder.hasGarage;
        this.hasSwimmingPool = builder.hasSwimmingPool;
        this.hasGarden = builder.hasGarden;
    }

    @Override
    public String toString() {
        return "House{" +
            "foundation='" + foundation + '\'' +
            ", structure='" + structure + '\'' +
            ", roof='" + roof + '\'' +
            ", hasGarage=" + hasGarage +
            ", hasSwimmingPool=" + hasSwimmingPool +
            ", hasGarden=" + hasGarden +
            '}';
    }

    public static class HouseBuilder{
        private String foundation;
        private String structure;
        private String roof;
        private boolean hasGarage;
        private boolean hasSwimmingPool;
        private boolean hasGarden;

        //Builder Constructor with mandatory parameters
        public HouseBuilder(String foundation,String structure,String
roof){
            this.foundation = foundation;
            this.structure = structure;
            this.roof = roof;
        }

        //Optional Parameters
        public HouseBuilder setGarden(boolean hasGarden){
            this.hasGarden = hasGarden;
            return this;
        }
        public HouseBuilder setSwimmingPool(boolean hasSwimmingPool){
            this.hasSwimmingPool = hasSwimmingPool;
        }
    }
}

```

```
        return this;
    }
    public HouseBuilder setGarage(boolean hasGarage){
        this.hasGarage = hasGarage;
        return this;
    }
    public House build(){
        return new House(this);
    }
}
}
```

```
package org.prateek.CreationalPatterns.BuilderPattern.Problem;

public class WithoutBuilderPattern {
    public static void main(String[] args) {
        House house = new
House("Concrete", "Wood", "Shingles", true, true, false);
        //House house2 = new House("Concrete", "Wood", "Shingles");
        // Constructor Explosion -> Too Many Constructors
        // Difficult to understand and maintain this code
        // this is where builder pattern comes into picture
        System.out.println(house);
    }
}
```

```
package org.prateek.CreationalPatterns.BuilderPattern.Problem;

public class House {
    private String foundation;
    private String structure;
    private String roof;
    private boolean hasGarage;
    private boolean hasSwimmingPool;
    private boolean hasGarden;

    //Constructor
    public House(String foundation,String structure,String roof, boolean
hasGarage,boolean hasSwimmingPool,boolean hasGarden){
        this.foundation = foundation;
        this.structure = structure;
        this.roof = roof;
        this.hasGarage = hasGarage;
        this.hasSwimmingPool = false;
        this.hasGarden = hasGarden;
    }

    @Override
    public String toString() {
        return "House{" +
            "foundation='" + foundation + '\\\'' +
            ", structure='" + structure + '\\\'' +
            ", roof='" + roof + '\\\'' +
            ", hasGarage=" + hasGarage +
            ", hasSwimmingPool=" + hasSwimmingPool +
            ", hasGarden=" + hasGarden +
            '}';
    }
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Solution;
```

```
public class TransportService {  
    public static void main(String[] args) {  
        //Direct create objects  
        Transport vehicle = TransportFactory.createTransport("car");  
        vehicle.deliver();  
    }  
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Solution;

public class TransportFactory {
    public static Transport createTransport(String type){
        switch(type.toLowerCase()){
            case "car":
                return new Car();
            case "bike":
                return new Bike();
            case "bus":
                return new Bus();
            default:
                throw new IllegalArgumentException("Unsupported transport
type");
        }
    }
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Solution;  
  
public interface Transport {  
    void deliver();  
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Solution;

public class Car implements Transport {
    @Override
    public void deliver() {
        System.out.println("Deliver by car");
    }
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Solution;

public class Bus implements Transport {
    @Override
    public void deliver() {
        System.out.println("Deliver by bus");
    }
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Solution;

public class Bike implements Transport {
    @Override
    public void deliver() {
        System.out.println("Deliver by bike");
    }
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Problem;
```

```
public class TransportService {  
    public static void main(String[] args) {  
        //Direct create objects  
        Transport car = new Car();  
        Transport bike = new Bike();  
        //add a bus object  
        Transport bus = new Bus();  
    }  
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Problem;  
  
public interface Transport {  
    void deliver();  
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Problem;
```

```
public class Car implements Transport{  
    @Override  
    public void deliver() {  
        System.out.println("Deliver by car");  
    }  
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Problem;
```

```
public class Bus implements Transport{  
    @Override  
    public void deliver() {  
        System.out.println("Deliver by bus");  
    }  
}
```

```
package org.prateek.CreationalPatterns.FactoryPattern.Problem;

public class Bike implements Transport{
    @Override
    public void deliver() {
        System.out.println("Deliver by bike");
    }
}
```

```
package org.prateek.CreationalPatterns.PrototypePattern.Solution;

public interface Prototype<T> {
    T clone();
}
```

```
package org.prateek.CreationalPatterns.PrototypePattern.Solution;
```

```
public class GamePiece implements Prototype<GamePiece>{
    public String getColor() {
        return color;
    }

    public int getPosition() {
        return position;
    }

    private String color;
    private int position;

    public void setColor(String color) {
        this.color = color;
    }

    public void setPosition(int position) {
        this.position = position;
    }

    public GamePiece(String color,int position){
        this.color = color;
        this.position = position;
    }

    @Override
    public String toString() {
        return "GamePiece{" +
            "color='" + color + '\'' +
            ", position=" + position +
            '\'';
    }

    @Override
    public GamePiece clone() {
        return new GamePiece(this.color,this.position);
    }
}
```

```
package org.prateek.CreationalPatterns.PrototypePattern.Solution;

public class GameClientWithPrototype {
    public static void main(String[] args) {
        GameBoard gameBoard = new GameBoard();
        gameBoard.addPiece(new GamePiece("Red", 1));
        gameBoard.addPiece(new GamePiece("Blue", 5));
        gameBoard.showBoardState();

        //Checkpoint this state
        GameBoard copiedBoard = gameBoard.clone();

        System.out.println("Copied Board");
        copiedBoard.showBoardState();
    }
}
```

```
package org.prateek.CreationalPatterns.PrototypePattern.Solution;

import java.util.ArrayList;
import java.util.List;

public class GameBoard implements Prototype<GameBoard> {
    private List<GamePiece> pieces = new ArrayList<>();

    public void addPiece(GamePiece piece) {
        pieces.add(piece);
    }

    public List<GamePiece> getPieces() {
        return pieces;
    }

    public void showBoardState() {
        for (GamePiece piece:pieces) {
            System.out.println(piece);
        }
    }

    @Override
    public GameBoard clone() {
        GameBoard newBoard = new GameBoard();
        for (GamePiece piece:pieces) {
            newBoard.addPiece(piece.clone());
        }
        return newBoard;
    }
}
```

```
package org.prateek.CreationalPatterns.PrototypePattern.Problem;
```

```
public class GamePiece {  
    public String getColor() {  
        return color;  
    }  
  
    public int getPosition() {  
        return position;  
    }  
  
    private String color;  
    private int position;  
  
    public void setColor(String color) {  
        this.color = color;  
    }  
  
    public void setPosition(int position) {  
        this.position = position;  
    }  
  
    public GamePiece(String color, int position){  
        this.color = color;  
        this.position = position;  
    }  
  
    @Override  
    public String toString() {  
        return "GamePiece{" +  
            "color='" + color + "'" +  
            ", position=" + position +  
            "}";  
    }  
}
```

```
package org.prateek.CreationalPatterns.PrototypePattern.Problem;

public class GameClientWithoutPrototype {
    public static void main(String[] args) {
        GameBoard gameBoard = new GameBoard();
        gameBoard.addPiece(new GamePiece("Red", 1));
        gameBoard.addPiece(new GamePiece("Blue", 5));
        gameBoard.showBoardState();

        //Checkpoint this state
        GameBoard copiedBoard = new GameBoard();
        for(GamePiece piece: gameBoard.getPieces()){
            copiedBoard.addPiece(new GamePiece(piece.getColor(),
piece.getPosition()));
        }

        System.out.println("Copied Board");
        copiedBoard.showBoardState();
    }
}
```

```
package org.prateek.CreationalPatterns.PrototypePattern.Problem;

import java.util.ArrayList;
import java.util.List;

public class GameBoard {
    private List<GamePiece> pieces = new ArrayList<>();

    public void addPiece(GamePiece piece) {
        pieces.add(piece);
    }

    public List<GamePiece> getPieces() {
        return pieces;
    }

    public void showBoardState() {
        for (GamePiece piece:pieces) {
            System.out.println(piece);
        }
    }
}
```

```
package org.prateek.CreationalPatterns.SingletonPattern.Solution;

public class SingletonPattern {
    public static void main(String[] args) {
        AppSettings appSettings = AppSettings.getInstance(); //change this
code
        AppSettings appSettingsCopy = AppSettings.getInstance();

        System.out.println(appSettingsCopy.getApiKey());
        System.out.println(appSettings.getApiKey());

        //More memory
        System.out.println(appSettings==appSettingsCopy);
    }
}
```

```
package org.prateek.CreationalPatterns.SingletonPattern.Solution;

public class AppSettings {
    // Step 1: Private static instance of the class
    private static AppSettings instance;

    private String databaseUrl;
    private String apiKey;

    //Step 2 : Private constructor to prevent direct object creation
    private AppSettings(){
        //Read settings from a config file
        databaseUrl = "jdbc:mysql://localhost:3306/mydatabase";
        apiKey = "12345-ABCDE";
    }

    //Step 3: Public static method to get the single instance
    public static AppSettings getInstance(){
        if(instance==null){
            instance = new AppSettings();
        }
        return instance;
    }

    public String getDatabaseUrl(){
        return databaseUrl;
    }
    public String getApiKey(){
        return apiKey;
    }
}
```

```
package org.prateek.CreationalPatterns.SingletonPattern.Problem;

public class WithoutSingletonPattern {
    public static void main(String[] args) {
        AppSettings appSettings = new AppSettings();
        AppSettings appSettingsCopy = new AppSettings();

        System.out.println(appSettingsCopy.getApiKey());
        System.out.println(appSettings.getApiKey());

        //More memory
        System.out.println(appSettings==appSettingsCopy);
    }
}
```

```
package org.prateek.CreationalPatterns.SingletonPattern.Problem;

public class AppSettings {
    private String databaseUrl;
    private String apiKey;

    public AppSettings() {
        //Read settings from a config file
        databaseUrl = "jdbc:mysql://localhost:3306/mydatabase";
        apiKey = "12345-ABCDE";
    }

    public String getDatabaseUrl() {
        return databaseUrl;
    }
    public String getApiKey() {
        return apiKey;
    }
}
```

```
package org.prateek.StructuralPatterns.AdapterPattern;
```

```
public class SendGridService {  
    public void sendEmail(String recipient, String title, String content){  
        System.out.println("Sending email via SendGrid to " + recipient);  
        System.out.println("Title: " + title);  
        System.out.println("Content: " + content);  
    }  
}
```

```
package org.prateek.StructuralPatterns.AdapterPattern;

public class SendGridAdapter implements NotificationService{
    private SendGridService sendGridService;

    public SendGridAdapter(SendGridService sendGridService){
        this.sendGridService = sendGridService;
    }

    @Override
    public void send(String to, String subject, String body) {
        //Adapter Method -> convert parameters and calls to SendGrid Method
        sendGridService.sendEmail(to,subject,body);
    }
}
```

```
package org.prateek.StructuralPatterns.AdapterPattern;

interface NotificationService {
    void send(String to,String subject,String body);
}
```

```
package org.prateek.StructuralPatterns.AdapterPattern;

//Legacy Code: Email Notification Service
public class EmailNotificationService implements NotificationService {
    public void send(String to,String subject,String body){
        System.out.println("Sending Email to " + to);
        System.out.println("Subject: " + subject);
        System.out.println("Body: " +body);
    }
}
```

```
package org.prateek.StructuralPatterns.AdapterPattern;

public class Client {
    public static void main(String[] args) {
        NotificationService emailService = new EmailNotificationService();
        emailService.send("customer@codingminutes.com", "order
confirmation", "your order has been received!");

        //NotificationService sendGridEmailService = new SendGridService();
        NotificationService emailServiceUsingSendGrid = new
SendGridAdapter(new SendGridService());
        emailServiceUsingSendGrid.send("customer@codingminutes.com", "order
confirmation", "your order has been received!");
    }
}
```

```
package org.prateek.StructuralPatterns.BridgePattern;

public class TV implements Device {
    private boolean isOn;
    private int volume;

    public TV() {
        this.isOn = false;
        this.volume = 0;
    }

    @Override
    public void powerOn() {
        this.isOn = true;
        System.out.println("TV is now ON");
    }

    @Override
    public void powerOff() {
        this.isOn = false;
        System.out.println("TV is now OFF");
    }

    @Override
    public void volumeUp() {
        if (isOn) {
            volume = Math.min(volume + 1, 100);
            System.out.println("TV volume increased to " + volume);
        } else {
            System.out.println("Turn on the TV to increase volume");
        }
    }

    @Override
    public void volumeDown() {
        if (isOn) {
            volume = Math.max(volume - 1, 0);
            System.out.println("TV volume decreased to " + volume);
        } else {
            System.out.println("Turn on the TV to decrease volume");
        }
    }

    @Override
    public void mute() {
        if (isOn) {
            volume = 0;
            System.out.println("TV is now muted");
        } else {
            System.out.println("Turn on the TV to mute");
        }
    }

    @Override
    public boolean isPoweredOn() {
```

```
        return isOn;
    }

    @Override
    public int getVolume() {
        return volume;
    }
}
```

```
package org.prateek.StructuralPatterns.BridgePattern;

public abstract class Remote {
    protected Device device;

    public Remote(Device device) {
        this.device = device;
    }

    public abstract void powerOn();
    public abstract void powerOff();
    public abstract void volumeUp();
    public abstract void volumeDown();
    public abstract void mute();

    public void setDevice(Device device) {
        this.device = device;
        System.out.println("Device changed to: " +
device.getClass().getSimpleName());
    }

    public Device getDevice() {
        return device;
    }
}
```

```

package org.prateek.StructuralPatterns.BridgePattern;

public class Radio implements Device {
    private boolean isOn;
    private int volume;

    public Radio() {
        this.isOn = false;
        this.volume = 0;
    }

    @Override
    public void powerOn() {
        this.isOn = true;
        System.out.println("Radio is now ON");
    }

    @Override
    public void powerOff() {
        this.isOn = false;
        System.out.println("Radio is now OFF");
    }

    @Override
    public void volumeUp() {
        if (isOn) {
            volume = Math.min(volume + 1, 50);
            System.out.println("Radio volume increased to " + volume);
        } else {
            System.out.println("Turn on the Radio to increase volume");
        }
    }

    @Override
    public void volumeDown() {
        if (isOn) {
            volume = Math.max(volume - 1, 0);
            System.out.println("Radio volume decreased to " + volume);
        } else {
            System.out.println("Turn on the Radio to decrease volume");
        }
    }

    @Override
    public void mute() {
        if (isOn) {
            volume = 0;
            System.out.println("Radio is now muted");
        } else {
            System.out.println("Turn on the Radio to mute");
        }
    }

    @Override
    public boolean isPoweredOn() {

```

```
        return isOn;
    }

    @Override
    public int getVolume() {
        return volume;
    }
}
```

```
package org.prateek.StructuralPatterns.BridgePattern;
```

```
public interface Device {  
    void powerOn();  
    void powerOff();  
    void volumeUp();  
    void volumeDown();  
    void mute();  
    boolean isPoweredOn();  
    int getVolume();  
}
```

```

package org.prateek.StructuralPatterns.BridgePattern;

public class Client {
    public static void main(String[] args) {
        Device tv = new TV();
        Device radio = new Radio();
        Device ac = new AC();

        Remote basicRemoteTV = new BasicRemote(tv);
        System.out.println("Basic Remote controlling TV");
        basicRemoteTV.powerOn();
        basicRemoteTV.volumeUp();
        basicRemoteTV.volumeUp();
        basicRemoteTV.mute();
        basicRemoteTV.powerOff();
        System.out.println();

        Remote basicRemoteRadio = new BasicRemote(radio);
        System.out.println("Basic Remote controlling Radio");
        basicRemoteRadio.powerOn();
        basicRemoteRadio.volumeUp();
        basicRemoteRadio.volumeDown();
        basicRemoteRadio.powerOff();
        System.out.println();

        AdvancedRemote advancedRemoteTV = new AdvancedRemote(tv);
        System.out.println("Advanced Remote controlling TV");
        advancedRemoteTV.powerOn();
        advancedRemoteTV.volumeUp();
        advancedRemoteTV.volumeUp();
        advancedRemoteTV.mute();
        advancedRemoteTV.displayStatus();
        advancedRemoteTV.unmute();
        advancedRemoteTV.powerOff();
        System.out.println();

        AdvancedRemote advancedRemoteAC = new AdvancedRemote(ac);
        System.out.println("Advanced Remote controlling AC");
        advancedRemoteAC.powerOn();
        advancedRemoteAC.volumeUp();
        advancedRemoteAC.volumeUp();
        advancedRemoteAC.displayStatus();
        advancedRemoteAC.volumeDown();
        advancedRemoteAC.powerOff();
        System.out.println();

        System.out.println("Runtime Flexibility: Switching Device");
        Device freshTV = new TV();
        Device freshRadio = new Radio();
        Device freshAC = new AC();

        Remote remote = new BasicRemote(freshTV);
        remote.powerOn();
        remote.volumeUp();
    }
}

```

```
        remote.setDevice(freshRadio);  
        remote.powerOn();  
        remote.volumeUp();  
  
        remote.setDevice(freshAC);  
        remote.powerOn();  
        remote.volumeUp();  
        System.out.println();  
    }  
}
```

```
package org.prateek.StructuralPatterns.BridgePattern;

public class BasicRemote extends Remote {

    public BasicRemote(Device device) {
        super(device);
    }

    @Override
    public void powerOn() {
        device.powerOn();
    }

    @Override
    public void powerOff() {
        device.powerOff();
    }

    @Override
    public void volumeUp() {
        device.volumeUp();
    }

    @Override
    public void volumeDown() {
        device.volumeDown();
    }

    @Override
    public void mute() {
        device.mute();
    }
}
```

```

package org.prateek.StructuralPatterns.BridgePattern;

public class AdvancedRemote extends Remote {
    private boolean isMuted;

    public AdvancedRemote(Device device) {
        super(device);
        this.isMuted = false;
    }

    @Override
    public void powerOn() {
        device.powerOn();
    }

    @Override
    public void powerOff() {
        device.powerOff();
    }

    @Override
    public void volumeUp() {
        if (!isMuted) {
            device.volumeUp();
        } else {
            System.out.println("Device is muted. Unmute first");
        }
    }

    @Override
    public void volumeDown() {
        if (!isMuted) {
            device.volumeDown();
        } else {
            System.out.println("Device is muted. Unmute first");
        }
    }

    @Override
    public void mute() {
        if (!isMuted) {
            device.mute();
            isMuted = true;
        } else {
            System.out.println("Device is already muted");
        }
    }

    public void unmute() {
        if (isMuted) {
            System.out.println("Unmuting device...");
            isMuted = false;
            if (device.getVolume() == 0) {
                device.volumeUp();
            }
        }
    }
}

```

```
    } else {  
        System.out.println("Device is not muted");  
    }  
}  
  
public void displayStatus() {  
    System.out.println("Device Status:");  
    System.out.println("Device Type: " +  
device.getClass().getSimpleName());  
    System.out.println("Powered On: " + device.isPoweredOn());  
    System.out.println("Volume/Temperature: " + device.getVolume());  
    System.out.println("Muted: " + isMuted);  
}  
}
```

```

package org.prateek.StructuralPatterns.BridgePattern;

public class AC implements Device {
    private boolean isOn;
    private int temperature;

    public AC() {
        this.isOn = false;
        this.temperature = 25;
    }

    @Override
    public void powerOn() {
        this.isOn = true;
        System.out.println("AC is now ON at temperature " + temperature);
    }

    @Override
    public void powerOff() {
        this.isOn = false;
        System.out.println("AC is now OFF");
    }

    @Override
    public void volumeUp() {
        if (isOn) {
            temperature = Math.min(temperature + 1, 30);
            System.out.println("AC temperature increased to " +
temperature);
        } else {
            System.out.println("Turn on the AC to increase temperature");
        }
    }

    @Override
    public void volumeDown() {
        if (isOn) {
            temperature = Math.max(temperature - 1, 16);
            System.out.println("AC temperature decreased to " +
temperature);
        } else {
            System.out.println("Turn on the AC to decrease temperature");
        }
    }

    @Override
    public void mute() {
        if (isOn) {
            temperature = 25;
            System.out.println("AC temperature reset to default: " +
temperature);
        } else {
            System.out.println("Turn on the AC to reset temperature");
        }
    }
}

```

```
@Override
public boolean isPoweredOn() {
    return isOn;
}

@Override
public int getVolume() {
    return temperature;
}
}
```

```
package org.prateek.StructuralPatterns.CompositePattern.Solution;

import java.util.ArrayList;
import java.util.List;

public class Folder implements FileSystemComponent{
    private String name;
    private List<FileSystemComponent> components = new ArrayList<>();

    public Folder(String name){
        this.name = name;
    }

    public void addComponent(FileSystemComponent component){
        components.add(component);
    }

    public void showDetails(){
        System.out.println("Folder: " + name);
        for(FileSystemComponent component:components){
            component.showDetails();
        }
    }
}
```

```
package org.prateek.StructuralPatterns.CompositePattern.Solution;

public interface FileSystemComponent {
    void showDetails();
}
```

```
package org.prateek.StructuralPatterns.CompositePattern.Solution;
```

```
public class FileSystemApp {  
    public static void main(String[] args) {  
        FileSystemComponent file1 = new File("File1.txt");  
        FileSystemComponent file2 = new File("File2.txt");  
        Folder folder = new Folder("Documents");  
        folder.addComponent(file1);  
        folder.addComponent(file2);  
  
        //Subfolder  
        Folder subfolder = new Folder("Subfolder");  
        FileSystemComponent file3 = new File("File3.txt");  
  
        subfolder.addComponent(file3);  
        folder.addComponent(subfolder);  
  
        folder.showDetails();  
    }  
}
```

```
package org.prateek.StructuralPatterns.CompositePattern.Solution;

public class File implements FileSystemComponent {
    private String name;

    public File(String name){
        this.name = name;
    }

    public void showDetails(){
        System.out.println("File : "+name);
    }
}
```

```
package org.prateek.StructuralPatterns.CompositePattern.Problem;

import java.util.ArrayList;
import java.util.List;

public class Folder {
    private String name;
    private List<File> files = new ArrayList<>();

    public Folder(String name){
        this.name = name;
    }

    public void addFile(File file){
        files.add(file);
    }

    public void showDetails(){
        System.out.println("Folder: " + name);
        for(File file:files){
            file.showDetails();
        }
    }
}
```

```
package org.prateek.StructuralPatterns.CompositePattern.Problem;
```

```
public class FileSystemApp {  
    public static void main(String[] args) {  
        File file1 = new File("File1.txt");  
        File file2 = new File("File2.txt");  
        Folder folder = new Folder("Documents");  
        folder.addFile(file1);  
        folder.addFile(file2);  
  
        folder.showDetails();  
    }  
}
```

```
package org.prateek.StructuralPatterns.CompositePattern.Problem;
```

```
public class File {  
    private String name;  
  
    public File(String name){  
        this.name = name;  
    }  
  
    public void showDetails(){  
        System.out.println("File : " +name);  
    }  
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Solution;
```

```
abstract class PizzaDecorator implements Pizza{  
    protected Pizza decoratedPizza;  
  
    public PizzaDecorator(Pizza pizza){  
        this.decoratedPizza = pizza;  
    }  
  
    @Override  
    public String getDescription() {  
        return decoratedPizza.getDescription();  
    }  
  
    @Override  
    public double getCost() {  
        return decoratedPizza.getCost();  
    }  
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Solution;

public class PizzaApp {
    public static void main(String[] args) {
        //Basic Pizza
        Pizza pizza = new BasicPizza();

        //Add Cheese
        pizza = new CheeseDecorator(pizza);
        pizza = new OliveDecorator(pizza);
        pizza = new MushroomDecorator(pizza);

        System.out.println(pizza.getDescription());
        System.out.println(pizza.getCost());
    }
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Solution;
```

```
public interface Pizza {  
    String getDescription();  
    double getCost();  
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Solution;
```

```
public class OliveDecorator extends PizzaDecorator {  
    public OliveDecorator(Pizza pizza){  
        super(pizza);  
    }  
  
    public String getDescription(){  
        return decoratedPizza.getDescription() + ", Olive";  
    }  
    public double getCost(){  
        return decoratedPizza.getCost() + 0.50;  
    }  
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Solution;

public class MushroomDecorator extends PizzaDecorator {
    public MushroomDecorator(Pizza pizza) {
        super(pizza);
    }

    public String getDescription() {
        return decoratedPizza.getDescription() + ", Mushroom";
    }

    public double getCost() {
        return decoratedPizza.getCost() + 2.00;
    }
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Solution;

public class CheeseDecorator extends PizzaDecorator {
    public CheeseDecorator(Pizza pizza){
        super(pizza);
    }

    public String getDescription(){
        return decoratedPizza.getDescription() + ", Cheese";
    }
    public double getCost(){
        return decoratedPizza.getCost() + 1.00;
    }
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Solution;

public class BasicPizza implements Pizza {
    @Override
    public String getDescription() {
        return "Basic Pizza";
    }

    @Override
    public double getCost() {
        return 5.00; //base cost
    }
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Problem;

public class PizzaApp {
    public static void main(String[] args) {
        Pizza pizza = new CheeseOlivePizza();
        System.out.println(pizza.getDescription());
        System.out.println(pizza.getCost());
    }
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Problem;
```

```
public interface Pizza {  
    String getDescription();  
    double getCost();  
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Problem;

public class CheesePizza extends BasicPizza{
    @Override
    public String getDescription() {
        return super.getDescription() + ", Cheese";
    }

    @Override
    public double getCost() {
        return super.getCost() + 1.00;
    }
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Problem;

public class CheeseOlivePizza extends CheesePizza{

    @Override
    public String getDescription() {
        return super.getDescription() + ", Olives";
    }

    @Override
    public double getCost() {
        return super.getCost() + 0.50;
    }
}
```

```
package org.prateek.StructuralPatterns.DecoratorPattern.Problem;

public class BasicPizza implements Pizza{
    @Override
    public String getDescription() {
        return "Basic Pizza";
    }

    @Override
    public double getCost() {
        return 5.00; //base cost
    }
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Solution;

public class Client {
    public static void main(String[] args) {
        // Client code interacting with API Gateway (Facade)
        APIGateway apiGateway = new APIGateway();
        //Task

        System.out.println(apiGateway.getFullOrderDetails("123", "456", "789"));
    }
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Solution;

public class APIGateway {
    UserService userService;
    OrderService orderService;
    PaymentService paymentService;

    public APIGateway() {
        this.userService = new UserService();
        this.orderService = new OrderService();
        this.paymentService = new PaymentService();
    }

    //Task
    public String getFullOrderDetails(String userId, String orderId, String
paymentId) {
        String userDetails = userService.getUserDetails(userId);
        String orderDetails = orderService.getOrderDetails(orderId);
        String paymentProcessing =
paymentService.processPayment(paymentId);

        return userDetails + "\n" + orderDetails + "\n" +
paymentProcessing;
    }
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Solution;
```

```
// Microservice for User Management
```

```
class UserService {  
    public String getUserDetails(String userId) {  
        // Simulate fetching user details  
        return "User details for userId: " + userId;  
    }  
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Solution;

// Microservice for Payment Processing
class PaymentService {
    public String processPayment(String paymentId) {
        // Simulate payment processing
        return "Processing payment with paymentId: " + paymentId;
    }
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Solution;

// Microservice for Order Management
class OrderService {
    public String getOrderDetails(String orderId) {
        // Simulate fetching order details
        return "Order details for orderId: " + orderId;
    }
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Problem;

// Microservice for User Management
class UserService {
    public String getUserDetails(String userId) {
        // Simulate fetching user details
        return "User details for userId: " + userId;
    }
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Problem;

// Microservice for Payment Processing
class PaymentService {
    public String processPayment(String paymentId) {
        // Simulate payment processing
        return "Processing payment with paymentId: " + paymentId;
    }
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Problem;
```

```
// Microservice for Order Management
```

```
class OrderService {  
    public String getOrderDetails(String orderId) {  
        // Simulate fetching order details  
        return "Order details for orderId: " + orderId;  
    }  
}
```

```
package org.prateek.StructuralPatterns.FacadePattern.Problem;

public class Client {
    public static void main(String[] args) {
        // Client code interacting with different services directly
        UserService userService = new UserService();
        OrderService orderService = new OrderService();
        PaymentService paymentService = new PaymentService();

        //Task
        System.out.println(userService.getUserDetails("123"));
        System.out.println(orderService.getOrderDetails("456"));
        System.out.println(paymentService.processPayment("789"));
    }
}
```

```
package org.prateek.StructuralPatterns.FlyWeightPattern.Solution;
```

```
public class Game {  
    public static void main(String[] args) {  
        // 5 Red Bullet Objects  
        for(int i=0;i<5;i++){  
            Bullet bullet = new Bullet("Red",i*10,i*12,5);  
        }  
        // 5 Green Bullet Objects  
        for(int i=0;i<5;i++){  
            Bullet bullet = new Bullet("Red",i*10,i*12,5);  
        }  
    }  
}
```

```
package org.prateek.StructuralPatterns.FlyWeightPattern.Solution;

import java.util.HashMap;

public class BulletTypeFactory {
    private static final HashMap<String,BulletType> bulletTypes = new
    HashMap<>();

    public static BulletType getBulletType(String color){
        if(!bulletTypes.containsKey(color)){
            bulletTypes.put(color, new BulletType(color));
        }
        return bulletTypes.get(color);
    }
}
```

```
package org.prateek.StructuralPatterns.FlyWeightPattern.Solution;

//Flyweight Class
public class BulletType {
    private String color; //Intrinsic Property

    public BulletType(String color){
        this.color = color;
        System.out.println("Creating bulletType with color " + color);
    }
}
```

```
package org.prateek.StructuralPatterns.FlyWeightPattern.Solution;

class Bullet {
    private BulletType type; // Intrinsic property shared by all bullets
    private int x, y;        // Extrinsic properties unique to each bullet
    private int velocity;

    public Bullet(String color, int x, int y, int velocity) {
        this.type = BulletTypeFactory.getBulletType(color);
        this.x = x;
        this.y = y;
        this.velocity = velocity;
        System.out.println("Creating bullet at (" + x + ", " + y + ") with
velocity " + velocity);
    }
    public void display() {
        System.out.println("Bullet at (" + x + ", " + y + ") moving at
velocity " + velocity);
    }
}
```

```
package org.prateek.StructuralPatterns.FlyWeightPattern.Problem;

public class Game {
    public static void main(String[] args) {
        // 5 Red Bullet Objects
        for(int i=0;i<5;i++) {
            Bullet bullet = new Bullet("Red",i*10,i*12,5);
        }
        // 5 Green Bullet Objects
        for(int i=0;i<5;i++) {
            Bullet bullet = new Bullet("Red",i*10,i*12,5);
        }
        //Problems
        // Memory Overhead : Every bullet stores redundant data like Color,
        Image
        // Performance : Slow performance when many bullets are created
    }
}
```

```
package org.prateek.StructuralPatterns.FlyWeightPattern.Problem;

class Bullet {
    private String color; // Intrinsic property shared by all bullets
    private int x, y;      // Extrinsic properties unique to each bullet
    private int velocity;

    public Bullet(String color, int x, int y, int velocity) {
        this.color = color; // Same for all bullets
        this.x = x;
        this.y = y;
        this.velocity = velocity;
        System.out.println("Creating bullet at (" + x + ", " + y + ") with
velocity " + velocity);
    }
    public void display() {
        System.out.println("Bullet at (" + x + ", " + y + ") moving at
velocity " + velocity);
    }
}
```

```
package org.prateek.StructuralPatterns.ProxyPattern.Solution;

public class RealImage implements Image {
    private String filename;
    public RealImage(String filename){
        this.filename = filename;
        loadImageFromDisk(); //Expensive Operation
    }

    private void loadImageFromDisk(){
        System.out.println("Loading image from disk " + filename);
    }

    @Override
    public void display() {
        System.out.println("Displaying: " +filename);
    }
}
```

```
package org.prateek.StructuralPatterns.ProxyPattern.Solution;

public class ProxyImage implements Image{
    private String filename;
    private RealImage realImage;//Proxy reference to real Image

    public ProxyImage(String filename){
        this.filename = filename;
    }
    @Override
    public void display() {
        if(realImage==null){
            realImage = new RealImage("filename"); //Image is loaded +
cached
        }
        realImage.display();
    }
}
```

```
package org.prateek.StructuralPatterns.ProxyPattern.Solution;  
  
public interface Image {  
    void display();  
}
```

```
package org.prateek.StructuralPatterns.ProxyPattern.Solution;

public class Client {
    public static void main(String[] args) {
        Image img1 = new ProxyImage("dog.png");
        Image img2 = new ProxyImage("cat.png");
        //The image is loaded lazily when needed,
        // saving time and resources if the
        // image is never displayed.
        img1.display();
        img1.display();

    }
}
```

```
package org.prateek.StructuralPatterns.ProxyPattern.Problem;

public class RealImage implements Image{
    private String filename;
    public RealImage(String filename){
        this.filename = filename;
        loadImageFromDisk(); //Expensive Operation
    }

    private void loadImageFromDisk(){
        System.out.println("Loading image from disk " + filename);
    }

    @Override
    public void display() {
        System.out.println("Displaying: " +filename);
    }
}
```

```
package org.prateek.StructuralPatterns.ProxyPattern.Problem;
```

```
public interface Image {  
    void display();  
}
```

```
package org.prateek.StructuralPatterns.ProxyPattern.Problem;

public class Client {
    public static void main(String[] args) {
        Image img1 = new RealImage("dog.png");
        Image img2 = new RealImage("cat.png");
        //Here, the RealImage is loaded every time we create it,
        // which can be inefficient if the image is not always required.
        img1.display();
        img1.display();

    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V1;
```

```
public class Vehicle {
```

```
    String numberPlate;
```

```
    String type;
```

```
    public Vehicle(String numberPlate, String type) {
```

```
        this.numberPlate = numberPlate;
```

```
        this.type = type;
```

```
    }
```

```
}
```

```

package org.prateek.LLDProject.RideSharingApp.V1;

import java.util.ArrayList;
import java.util.List;

public class RideSharingAppService {
    //Matching Service
    private List<Driver> drivers = new ArrayList<>();
    private List<Passenger> passengers = new ArrayList<>();

    //Methods to add Drivers and Passengers
    public void addDriver(Driver driver){
        drivers.add(driver);
    }
    public void addPassenger(Passenger passenger){
        passengers.add(passenger);
    }

    //Booking Ride
    public void bookRide(Passenger passenger,double distance){
        //corner case
        if(drivers.isEmpty()){
            System.out.println("No drivers are available for " +
passenger.name);
            return;
        }
        // Hard-coded assignment logic
        // find the nearest driver
        // O(N) Brute Force
        Driver assignedDriver = null;
        double minDistance = Double.MAX_VALUE;

        for(Driver driver:drivers){
            double currentDriverDistance =
calcDistance(passenger.location,driver.location);
            if(currentDriverDistance<minDistance){
                minDistance = currentDriverDistance;
                assignedDriver = driver;
            }
        }
        //Fare Calculation
        double expectedFare =
calcFare(assignedDriver.getVehicle(),distance);

        // Track the driver state - BOOKED/AVAILABLE

        // Show the driver and fare to the passenger
        System.out.println("Ride booked for "+passenger.name + " with
driver " + assignedDriver.getName() +" for a fare of " + expectedFare);
        System.out.println("Driver is on the way " +
assignedDriver.getName());
    }

    private double calcFare(Vehicle vehicle,double distance){
        if(vehicle.type.equals("Car")){

```

```
        return distance*20;
    }
    else if(vehicle.type.equals(10)){
        return distance*10;
    }
    else{
        return distance*8; //hard-code value
    }
}

private double calcDistance(Location one, Location two){
    //Euclidean Distance
    double dx = one.getLatitude() - two.getLatitude();
    double dy = one.getLongitude() - two.getLongitude();
    return Math.sqrt(dx*dx + dy*dy);
}
}
```

```
package org.prateek.LLDProject.RideSharingApp.V1;
```

```
public class Passenger {  
    String name;  
    Location location;  
  
    public Passenger(String name, Location location) {  
        this.name = name;  
        this.location = location;  
    }  
}
```

```
package org.prateek.LLDProject.RideSharingApp.V1;
```

```
public class Location {  
    private double latitude;  
    private double longitude;  
    public Location(double latitude, double longitude) {  
        this.latitude = latitude;  
        this.longitude = longitude;  
    }  
    public double getLatitude() {  
        return latitude;  
    }  
  
    public double getLongitude() {  
        return longitude;  
    }  
}
```

```
package org.prateek.LLDProject.RideSharingApp.V1;

public class Driver {

    private String name;
    private Vehicle vehicle;
    Location location;

    public Driver(String name, Location location, Vehicle vehicle) {
        this.name = name;
        this.vehicle = vehicle;
        this.location = location;
    }

    public Location getLocation() {
        return location;
    }

    public void setLocation(Location location) {
        this.location = location;
    }

    public String getName() {
        return name;
    }

    public Vehicle getVehicle() {
        return vehicle;
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V1;

public class Client {
    public static void main(String[] args) {
        Location loc1 = new Location(12.9716, 77.5946); // Bangalore
        Location loc2 = new Location(12.9352, 77.6245); // Near Bangalore
        Location loc3 = new Location(13.0352, 77.6175); // Another place
        near Bangalore

        // Create Vehicles
        Vehicle car = new Vehicle("AB123CD", "Car");
        Vehicle bike = new Vehicle("XY987Z", "Bike");

        // Create Drivers
        Driver driver1 = new Driver("Alice", loc2, car);
        Driver driver2 = new Driver("Bob", loc3, bike);

        // Create Passengers
        Passenger passenger1 = new Passenger("John", loc1);
        Passenger passenger2 = new Passenger("Rahul", loc3);

        // Ride Sharing App
        RideSharingAppService app = new RideSharingAppService();
        app.addDriver(driver1);
        app.addDriver(driver2);
        app.addPassenger(passenger1);
        app.addPassenger(passenger2);

        //Book the Ride
        app.bookRide(passenger1,10);
        app.bookRide(passenger2,20);
        app.bookRide(passenger2,20);
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

public class Driver extends User{
    private Vehicle vehicle;
    public Driver(String name, String email, Location location, Vehicle
vehicle) {
        super(name, email, location);
        this.vehicle = vehicle;
    }
    public Vehicle getVehicle(){
        return vehicle;
    }
    //Any other methods??
    public void notify(String msg){
        System.out.println("Driver " + msg);
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

public class Client {
    public static void main(String[] args) {
        Location loc1 = new Location(12.9716, 77.5946); // Bangalore
        Location loc2 = new Location(12.9352, 77.6245); // Near Bangalore
        Location loc3 = new Location(13.0352, 77.6175); // Another place
        near Bangalore

        Vehicle car = new Car("AB123CD");
        Vehicle bike = new Bike("XY987Z");

        Driver driver1 = new Driver("Alice", "alice@rideshare.com", loc2,
car);
        Driver driver2 = new Driver("Bob", "bob@rideshare.com", loc3,
bike);

        Passenger passenger1 = new Passenger("John", "john@rideshare.com",
loc1);

        RideMatchingSystem rideMatchingSystem = new RideMatchingSystem();
        rideMatchingSystem.addDriver(driver1);
        rideMatchingSystem.addDriver(driver2);

        rideMatchingSystem.requestRide(passenger1, 10, new
StandardFareStrategy());
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

;

public class Car extends Vehicle {
    public Car(String numberPlate) {
        super(numberPlate);
    }

    @Override
    public double getFarePerKm() {
        return 20;
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

public class Bike extends Vehicle {
    public Bike(String numberPlate) {
        super(numberPlate);
    }

    @Override
    public double getFarePerKm() {
        return 10;
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;
```

```
abstract class Vehicle {  
    protected String numberPlate;  
  
    public Vehicle(String numberPlate){  
        this.numberPlate = numberPlate;  
    }  
    //Fare Calc  
    public abstract double getFarePerKm();  
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

import org.prateek.LLDProject.RideSharingApp.V1.Passenger;

abstract class User {
    protected String name;
    protected String email;
    protected Location location;

    public User(String name,String email,Location location){
        this.name = name;
        this.email = email;
        this.location = location;
    }
    //Setter
    public Location getLocation(){
        return location;
    }
    public void setLocation(Location location){
        this.location = location;
    }
    //any other methods ?
    public abstract void notify(String msg);
}
```

```

package org.prateek.LLDProject.RideSharingApp.V2;

import java.util.ArrayList;
import java.util.List;

public class RideMatchingSystem {
    private List<Driver> availableDrivers = new ArrayList<>();

    public void addDriver(Driver driver){
        availableDrivers.add(driver);
    }

    public void requestRide(Passenger passenger, double
distance, FareStrategy fareStrategy){
        //base case
        if(availableDrivers.isEmpty()){
            //mechanism to notify the passenger
            passenger.notify("No drivers are unavailable");
            return;
        }
        //find the nearest driver available
        Driver nearestDriver = findNearestDriver(passenger.getLocation());

        //Mediator
        availableDrivers.remove(nearestDriver);
        // passenger.notify("Ride schedule successfully" + nearestDriver);
        Ride ride = new
Ride(passenger, nearestDriver, distance, fareStrategy);
        //Calc fare
        ride.calculateFare();

        passenger.notify("Ride schedued with fare + Rs"+ride.getFare());
        nearestDriver.notify("You have a new ride request for
"+ride.getFare());

        //Change the Status of the Ride
        ride.updateStatus(RideStatus.ONGOING);

        //Change the status of ride after ride is finished
        ride.updateStatus(RideStatus.COMPLETED);
        availableDrivers.add(nearestDriver);
        return;
    }

    private Driver findNearestDriver(Location passengerLocation){
        Driver assignedDriver = null;
        double minDist = Double.MAX_VALUE;

        for(Driver driver:availableDrivers){
            double distance =
driver.getLocation().calcDistance(passengerLocation);
            if(distance<minDist){
                minDist = distance;
            }
        }
        return assignedDriver;
    }
}

```

```
        assignedDriver = driver;
    }
    }
    return assignedDriver;
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

enum RideStatus{
    SCHEDULED, ONGOING, COMPLETED;
}

public class Ride {
    private Passenger passenger;
    private Driver driver;
    private double distance;
    private FareStrategy fareStrategy;
    private double fare;
    private RideStatus status;

    public Ride(Passenger passenger, Driver driver, double distance,
FareStrategy fareStrategy){
        this.passenger = passenger;
        this.driver = driver;
        this.distance = distance;
        this.fareStrategy = fareStrategy;
        this.status = RideStatus.SCHEDULED;
    }

    public void calculateFare(){
        this.fare = fareStrategy.calcFare(driver.getVehicle(), distance);
    }

    public void updateStatus(RideStatus status){
        this.status = status;
        notifyUsers(status);
    }
    private void notifyUsers(RideStatus status){
        passenger.notify("Your ride is "+status);
        driver.notify("Ride Status : "+status);
    }

    public double getFare(){
        return fare;
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

public class Passenger extends User{
    public Passenger(String name, String email, Location location) {
        super(name, email, location);
    }
    //any other methods??
    public void notify(String msg){
        System.out.println("passenger : " + msg);
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

public class Location {
    private double latitude;
    private double longitude;
    public Location(double latitude, double longitude) {
        this.latitude = latitude;
        this.longitude = longitude;
    }
    public double getLatitude() {
        return latitude;
    }

    public double getLongitude() {
        return longitude;
    }

    public double calcDistance(Location two){
        //Euclidean Distance
        double dx = this.getLatitude() - two.getLatitude();
        double dy = this.getLongitude() - two.getLongitude();
        return Math.sqrt(dx*dx + dy*dy);
    }
}
```

```
package org.prateek.LLDProject.RideSharingApp.V2;

public interface FareStrategy {
    double calcFare(Vehicle vehicle, double distance);
}

class StandardFareStrategy implements FareStrategy{

    @Override
    public double calcFare(Vehicle vehicle, double distance) {
        return vehicle.getFarePerKm()*distance;
    }
}

class SharedFareStrategy implements FareStrategy{

    @Override
    public double calcFare(Vehicle vehicle, double distance) {
        return vehicle.getFarePerKm()*distance*.50;
    }
}

class LuxuryStrategy implements FareStrategy{

    @Override
    public double calcFare(Vehicle vehicle, double distance) {
        //additional 50% surcharge for luxury ride
        return vehicle.getFarePerKm()*distance*1.5;
    }
}
```